



Quasi oppositional Aquila optimizer-based task scheduling approach in an IoT enabled cloud environment

M. Kandan¹ · Anbazhagan Krishnamurthy² · S. Arun Mozhi Selvi³ · Mohamed Yacin Sikkandar⁴ · Mohamed Abdelkader Aboamer⁴ · T. Tamilvizhi⁵

Accepted: 6 January 2022

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2022

Abstract

Large-scale applications of the Internet of Things (IoT) necessitate significant computing tasks and storage resources that are progressively installed in the cloud environment. Related to classical computing models, the features of the cloud, such as pay-as-you-go, indefinite expansions, and dynamic acquisition, signify various services to these applications utilizing the IoT structure. A major challenge is to fulfill the quality of service necessities but schedule tasks to resources. The resource allocation scheme is affected by different undefined reasons in real-time platforms. Several works have considered the factors in the design of effective task scheduling techniques. In this context, this research addresses the issue of resource allocation and management in an IoT-enabled CC environment by designing a novel quasi-oppositional Aquila optimizer-based task scheduling (QOAO-TS) technique. The QOAO technique involves the integration of quasi-oppositional-based learning with an Aquila optimizer (AO). The traditional AO is stimulated by Aquila's behavior while catching the prey, and the QOAO is derived to improve the performance of the AO. The QOAO-TS technique aims to fulfill the makespan by accomplishing the optimum task scheduling process. The proposed QOAO-TS technique considers the relationship among task scheduling and satisfies the client's needs by minimizing the makespan. A wide range of simulations take place, and the results are investigated in terms of the span, throughput, flow time, lateness, and utilization ratio.

Keywords Cloud computing · Internet of Things · Task scheduling · Objective function · Makespan · Bioinspired algorithm

✉ M. Kandan
kandan.win25@gmail.com

Extended author information available on the last page of the article

1 Introduction

In recent times, the Internet of Things (IoT) has become one of the primary revolutions in the information and communication technology fields. Using the IoT, internet connections expand beyond conventional smart devices such as tablets and smartphones to a wide variety of things and devices (vehicles, sensors, machines, and so on.) [1] for accomplishing various services and applications, such as medical treatment, health care, energy management, vehicle networks, traffic control, and so on. This wide range of application produces a large amount of information that should be analyzed, stored, and processed to attain useful data for meeting the user's needs and goals. Additionally, the scale and number of services and applications are also rapidly increasing, which requires processing ability that even effective smart devices cannot meet presently [2]. Cloud environments, i.e., called a huge resource center that enables ubiquitous access to provide and share resources to the user flexibly via virtualization methods, could be a possible platform for supporting IoT development. The restriction of present smart devices (processing power, lifetime of the battery, network resources, and memory space) could be minimized by taking resource-intensive and time-consuming tasks to effective computing platforms such as cloud computing when leaving the simple task for smart devices to manage [3]. Figure 1 illustrates the general structure of the IoT-enabled cloud environment.

Currently, many IoT services and devices can interact and connect with one another in a small business/single home at any time [4]. In addition, the essential QoS for users to access cloud services is increasing dramatically. In addition to scalability and context awareness, resource heterogeneity factors could make resource management a difficult process across the cloud. Scholars declare that the usage of IoT systems as a framework could make huge differences against the present conventional practice in CC models. Considering that the resource allocation mechanism in the CC model requires a sophisticated strategy for allocating better resources when fulfilling the user's requirements and demands, incorporating the IoT and CC is not

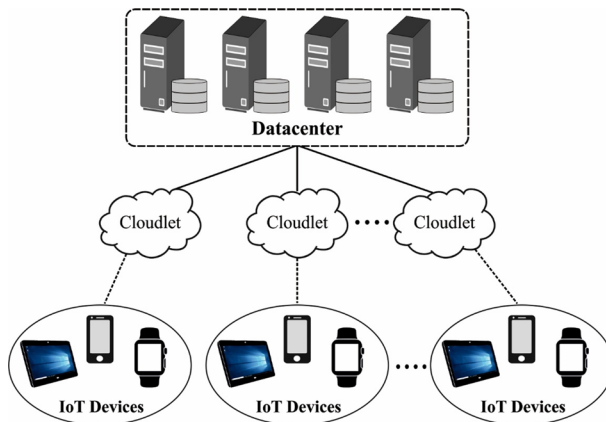


Fig. 1 General framework of IoT enabled cloud environment

an easier process, and it can be challenging to maintain, establish, and distribute different applications when meeting the peak demand of the user with a retained QoS requirement [5]. Due to the different abilities of the available heterogeneous devices and servers, multiple task scheduling processes are needed to meet the user's QoS requirement. This procedure is based on the flexibility, priority of the data traffic class, and allocation policy types. Indeed, there are several classes of experienced data traffic in the IoT paradigm, which also necessitates distinct QoS limitations that should be addressed when increasing the available cloud resource consumption [6]. Thus, it is vital to suggest a suitable scheduling method for providing optimum queue management to multiclass data traffic to ensure the highest possible consumption of IoT resource allocations among personal devices and heterogeneous systems of servers. There might be cases where standard scheduling algorithms in CC paradigms could not fulfill the QoS constraint. Additionally, this ineffective scheduling may cause low network throughput and undesired lengthy delays.

Since the IoT application task is heterogeneous in nature, load balancing and effective scheduling of IoT applications are essential [7]. Generally, the scheduling task is advanced by a heuristic technique that provides optimal and constructive solutions in polynomial time considering the imposed constraint. It searches paths in the solution space at the expense of neglecting a few potential paths [8]. The list scheduling classification of the scheduling task algorithm utilizes task queues and lists to allocate priority values to all tasks according to certain criteria. This algorithm is highly effective since the solution space is narrowed. However, this algorithm is based largely on heuristic parameters.

This study proposes a novel task scheduling technique in an IoT cloud environment using the quasi oppositional Aquila optimizer-based task scheduling (QOAO-TS) technique. The QOAO technique involves the integration of quasi-opposition-based learning (QOBL) with an aqueous optimizer (AO) to enhance the efficiency of the AO. The QOAO-TS technique aims to fulfill the makespan by accomplishing the optimum task scheduling process. Moreover, the QOAO-TS technique has utilized the relativity among the task scheduling and client need satisfaction by minimizing the makespan. To showcase the improved performance of the QOAO-TS technique over the other existing techniques, a detailed comparative analysis is carried out.

2 Related works

Feller et al. [9] considered scheduling tasks in cloud fog computing environments, and they proposed multi-agent-based models that aim to serve the essential task in the beginning, considering the task's priority, its status, wait time, and the resources needed to accomplish it effectively. Furthermore, in the scheduling task, initially suggest an upgrade to the priority values of the task when considering its dependency on other priorities and tasks. In Sheng et al. [10], task scheduling problems have been investigated in EC scenarios, and several tasks are scheduled to VM organized at the edge server by increasing the LTSD. The problems are created as an MDP in which the reward, state, state transition, and action are developed. They leverage DRL to resolve scheduling time (viz., task implementation order) and resource

allocations (viz., VM to which tasks are allocated), considering the variety of tasks and the heterogeneity of obtainable resources.

Hasan and Al-Rizzo [11] proposed a task scheduling model on the basis of the CPSO method to resolve the problems of resource management and allocation in heterogeneous and homogeneous IoT cloud computing. The aim is to fulfill the Makespan by implementing optimum scheduling tasks when taking into account the distinct policies of incoming tasks. In Abdel-Basset et al. [12], an energy-aware method based on the MPA is presented to tackle the TSFC model for improving the QoSs needed by the user. In addition to the typical MPA, they presented another 2 versions. The initial versions are named MMPA, which would modify MPA to enhance their exploitation ability through the last upgraded position rather than the last optimal position.

Al-Turjman et al. [13] introduced a task scheduling algorithm on the basis of strong CPSO and FIPS methods to resolve the problems of RA in heterogeneous and homogeneous cloud-based IoT applications. The aim is to fulfill the quality of services based on delay and throughput by implementing the optimum scheduling task when taking into account the distinct experienced data traffic classifications. Ma et al. [14] proposed a cost-aware and deadline scheduling method that reduces the implementation cost of a task under deadline constraints in IaaS models. Considering the VM acquisition delay and performance variation, the task is initially split into distinct levels based on the topological structure; thus, no dependencies exist among the tasks at similar levels. The 3 strings are employed for coding the gene in the presented model to better reflect the resilient and heterogeneous features of the cloud environment. Next, HEFT is employed to generate the individual with minimal cost and completion time.

In Basu et al. [15], an intelligent/cognitive model of a bioinspired algorithm is employed to find an optimum solution for the scheduling task for IoT applications in heterogeneous multi-processor cloud environments. The natural selection of genes and evolutionary foraging traits has demonstrated that the appropriate species survive naturally [16–19]. In these cases, appropriate scheduling is considered effective and follows the order of the task in multiprocessor environments. A hybrid method GAACO integrating GA and ACO has been employed for selecting optimal combinations of tasks at every phase.

3 The proposed model

As preemption is taken into account as a major factor in real-time systems for instantly serving the incoming task, nonpreemption presents further blocking time in various scenarios. Thus, the question arises as to how to develop a proper scheduler that performs a preemptive incoming task that requires an urgent service with minimal scheduling Makespan. Particularly, they could design the optimization objectives as scheduling the stream of huge requests or tasks arriving at the systems; thus, the maximal completion time of incoming tasks is minimalized. To evaluate the schedule, the performance optimality condition is employed as follows:

$$\text{Schedule length (Makespan)} C_{\max} = \max C_l, \quad (1)$$

whereas mean flow time is determined by

$$\bar{F}_{Task_j} = \frac{1}{n} \sum_{j=1}^n F_j, \quad (2)$$

and meanwhile, maximum lateness is

$$L_{\max} = \max L_l. \quad (3)$$

The features of VM are dedicated/parallel. The 3 kinds of parallel VMs are distinguished based on the time needed to process the tasks $Task_j$ on a VM, i.e., represented as p_l . When each VM has equivalent task processing time p_l , it can be named identical; however, when the task processing time is constant, they are named uniform. In the case of *IoT* assisted smart task applications, they consider vectors of processing time that describe the processing time requirement of certain tasks containing single applications $P_{Task_j} = [P_{task_{j1}}, P_{task_{j2}}, \dots, P_{task_{jTask_j}}]^T$, whereas $Task_j = \{task_{j1}, task_{j2}, \dots, task_{jTask_j}\}$; thus, $b_{task_{jTask_j}}$ represents the processing time of $task_{jTask_j}$ on the respective VM. Thus, the scheduling problems are determined by the set of variables defined in decision versions based on the scheduling process, which constructs a schedule for a provided problem. In this study, the QOAO-TS technique is derived for scheduling tasks in an *IoT* cloud environment.

Generally, more than one newborn lives to fledge in approximately twelve weeks. Because of its hunting bravery, Aquila is the most popular bird around the world. Male Aquila capture considerably more prey while solo-hunting. Aquila is one of the most skillful and intelligent hunters, possibly next after humans. The primary motivation for the presented AO method is acquired from the abovementioned method. The subsequent subsection describes how this process is modeled in *AO*.

In *AO*, it can be a population-based method, and the optimized rule begins with the population of candidate solutions (X), as shown in Eq. (4), i.e., made stochastically among the upper bound (*UB*) and lower bound (*LB*) of the provided problems. Thus far, the optimally attained solution is defined as the best solution in nearly all iterations [20].

$$x = \begin{bmatrix} x_{1,1} & \cdots & x_{1,j} & x_{1,Dim-1} & x_{1,Dim} \\ x_{2,1} & \cdots & x_{2,j} & \cdots & x_{2,Dim} \\ \cdots & \cdots & x_{i,j} & \cdots & \cdots \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ x_{N-1,1} & \cdots & x_{N-1,j} & \cdots & x_{N-1,Dim} \\ x_{N,1} & \cdots & x_{N,j} & x_{N,Dim-1} & x_{N,Dim} \end{bmatrix} \quad (4)$$

where X represents a set of present candidate solutions, i.e., arbitrarily generated in Eq. (5), X_i denotes the decision value (position) of the i^{th} solutions, N indicates the overall number of candidate solutions (population), and Dim denotes the dimensions of the problem.

$$X_{ij} = rand \times (UB_j - LB_j) + LB_j, i = 1, 2, \dots, Nj = 1, 2, \dots, Dim \quad (5)$$

where $rand$ denotes an arbitrary value, LB_j represents the j^{th} lower bound, and UB_j indicates the j^{th} upper bound of the provided problems.

The presented AO model simulates Aquila's behaviors at the time of hunting while presenting the action of all steps of the hunt. Therefore, the optimization procedure of the presented AO model can be given in 4 procedures: select the search space with higher soar using vertical stoop, explore within a different search space through contour flight using short glide attacks, exploit in a converge search space through lower flight using slower descent attacks, and swoop through walk and grab prey. The AO model could move in the exploration to exploitation step via distinct behavior according to these conditions when $t \leq \left(\frac{2}{3}\right) * T$ the exploration step would be excited; otherwise, the exploitation step would be performed. They modeled Aquila behavior as an arithmetical optimization model and determined an optimal solution subject to certain constraints. The arithmetical models of the AO are presented in the following.

In the expanded exploration process (X_1), the Aquila identifies the prey region and chooses the optimal hunting region through high. Now, the AO broadly explorer from higher soar to define the region of the search area. These behaviors are arithmetically shown in Eq. (6).

$$X_1(t+1) = X_{best}(t) \times \left(1 - \frac{t}{T}\right) + (X_M(t) - X_{best}(t) * rand), \quad (6)$$

where $X_1(t+1)$ represents the solution of the following iteration of t , i.e., created using the initial search process (X_1). $X_{best}(t)$ denotes the optimally attained solutions until iteration t^{th} and reflects the accurate location of the prey. Such equations $\left(\frac{1-t}{T}\right)$ are employed to control the extended search (exploration) by the number of iterations. $X_M(t)$ represents the location mean values of the present solution linked at the t^{th} iteration, i.e., estimated by Eq. (7). $rand$ represents an arbitrary number in the range of zero & one. t & T present the existing iteration and the maximal number of iterations, respectively.

$$X_M(t) = \frac{1}{N} \sum_{i=1}^N X_j(t), \forall j = 1, 2, \dots, Dim \quad (7)$$

where Dim represents the dimensional size of the issue and N indicates the candidate solution size. In the 2nd process (X_2), once the prey region is established from a higher soar, the Aquila circle above the targeted prey prepares the property and later attacks. These behaviors are arithmetically shown in Eq. (8).

$$X_2(t+1) = X_{best}(t) \times Levy(D) + X_R(t) + (y - x) * rand, \quad (8)$$

where $X_2(t+1)$ represents the solution of r , i.e., created using the 2nd search process (X_2). D is the dimension space, and $Levy(D)$ represents the Levy flight distribution. $X_R(t)$ Will an arbitrary solution in the interval of $[1N]$ at the i^{th} iteration.

$$Levy(D) = s \times \frac{u \times \sigma}{|v|^{\frac{1}{\beta}}} \quad (9)$$

where s represents a constant value set to 0.01, u , and v indicates arbitrary numbers among zero and one. σ is evaluated by Eq. (10).

$$\sigma = \left(\frac{\Gamma(1 + \beta) \times \sin\left(\frac{\pi\beta}{2}\right)}{\Gamma\left(\frac{1+\beta}{2}\right) \times \beta \times 2\left(\frac{\beta-1}{2}\right)} \right)^{\frac{1}{\beta}} \quad (10)$$

whereas β represent constant values set to 1.5. In Eq. (8), y and χ are employed to present a spiral form in the search, which is evaluated by.

$$y = r \times \cos(\theta) \quad (11)$$

$$x = r \times \sin(\theta) \quad (12)$$

whereas

$$r = r_1 + U \times D_1 \quad (13)$$

$$\theta = -\omega \times D_1 + \theta_1 \quad (14)$$

$$\theta_1 = \frac{3 \times \pi}{2} \quad (15)$$

r_1 takes the values among one and twenty to fix the number of search cycles, and U indicates smaller values set to 0.00565. D_1 denotes an integer number from one to the length of search space (Dim), and ω represents smaller values set to 0.005.

In the 3rd process (X_3), once the prey region is precisely stated and the Aquila is prepared for the attack and landing, the Aquila descends vertically with primary attacks to determine the prey response. This process is named lower flight using slower descent attacks. Now, AO utilizes the elected region of the targets to get closer to prey and attacks. These behaviors are arithmetically shown in Eq. (13).

$$X_3(t+1) = (X_{best}(t) - X_M(t)) \times \alpha - rand + ((UB - LB) \times rand + LB) \times \delta, \quad (16)$$

where $X_3(r+1)$ represents the solution of the following iteration of r , i.e., created using the 3rd search process (X_3). $X_{best}(t)$ represents the accurate place of the prey until iteration i^{th} (the optimally gained solutions), and $X_M(t)$ signifies the mean values of the present solutions at iteration r^{th} , i.e., evaluated by Eq. (7). $rand$ represent arbitrary values among zero & one. α and δ represent the exploitation adjustment parameter set in this work for smaller values (0.1). LB represents the lower bound, and UB represents the upper bound of the provided problems.

In the 4th process (X_4), once the Aquila get closer to prey, the Aquila attack the prey across the land-based on their stochastic movement. These behaviors are arithmetically shown in Eq. (17).

$$X_4(t+1) = QF \times X_{best}(t) - (G_1 \times X(t) \times rand) - G_2 \times Levy(D) + rand \times G_1, \quad (17)$$

where $X_4(t+1)$ represent the solutions of the following iteration of t , i.e., created using the 4th search process (X_4). QF represents a quality function employed to equilibrate the search strategy.

During the case 1D search space, a quasi-opposite number (qox) has been supposed to be an arbitrary value among the middle of search space $\frac{a+b}{2}$ but opposite number (ox), as provided in Eq. (18)

$$qox = rand\left(\left(\frac{a+b}{2}\right), ox\right) \quad (18)$$

Afterward, for the d-dimension search space, qox of X is considered to be $QOX = (qox_1, qox_2, \dots, qox_d)$, and the mathematical function in Eq. (19) is provided as:

$$qox_i = rand\left(\left(\frac{a_i + b_i}{2}\right), ox_i\right) \quad (19)$$

The qox point comprises maximal possibilities of being closer to the global optimum related to the opposite point, so QOBL has been extremely appropriate for enhancing the convergence rate. The fundamental populations of classic meta-heuristic manners are created in arbitrary approaches with no advanced knowledge of solution space. At this point, QOBL depends on the initialized population to realize a fitter candidate solution as concurrent assumptions of arbitrarily formed primary places and qox places maximizing the supremacy of the primary population and inspire the exploration model by exploring the effectual region of the search space.

The objectives of makespan are to consider the architecture of assigning a number of tasks indicated by $Task_J = \{task_{j1}, task_{j2}, \dots, task_{jTask_J}\}$ of J independent tasks to be treated on a set of VMs represented by $VM = VM_1, VM_2, \dots, VM_l$. Every user requests an application from distinct AEPs. They state that all VMs are similar to the 4 lower and upper layers of the cloud framework, i.e., determined by the request manager, user interface, external cloud interface, and scheduler. Assume that $CP = CP_1, CP_2, \dots, CP_n$ denotes a set of cloud suppliers. Given that CP_1 denotes the private cloud and $CP_2, \dots, CP_n$ represents the external cloud, (ECs). $App = a_1, a_2, \dots, a_w$ represents a set of applications. All the applications $a_j (j \in \{1, 2, \dots, w\})$ include parallel sets of an independent task with strict timing parameters based on release time, processing time, Due Date, and Deadline. Each request is anticipated to meet the IoT framework; however, this entry point might be physical devices such as smart devices, i.e., a portion of IoT, or they might be computer running web portals such as user interfaces, VM, or gateways. All the tasks must be performed on one or partitioned on several VMs, whereas the time needed to process tasks $Task_j$ on VM_1 is represented as b_l . The subsets of tasks mapped into VM_1 in schedule are represented by S_{VM1} . The represent adjacent

matrix $A = (a_{ij})$ for scheduling process $a_{ij} := |E_{ij}|$. When tasks begin to run, they should be accomplished based on the list scheduling method. Furthermore, the adjacent matrix $A = (a_{ij})$ defines precedence relations among the tasks for scheduling process $a_{ij} := |E_{ij}|$, whereas

$$E_J = l \begin{cases} \{e_l \leq e_J, \text{ if precedence from task}_J \text{ to task}_l\} \\ \{e_l \geq e_J, \text{ if successor from task}_l \text{ to task}_J\} \end{cases} \quad (20)$$

The mapping condition adjacency matrix $|G|$ represented by Π ensures the successor and precedence constraint values among $task_l$ and $task_J$, determined by

$$\Pi_{iJ} = \begin{pmatrix} task_{11} & task_{12} & \cdots & task_{1|J|} \\ task_{21} & task_{22} & \cdots & task_{2|J|} \\ task_{31} & task_{32} & \cdots & task_{3|J|} \\ \vdots & \vdots & \ddots & \vdots \\ task_{|I|1} & task_{|I|2} & \cdots & task_{|I||J|} \end{pmatrix} \quad (21)$$

whereas

$$\chi_{(I,J)} = \iota, \begin{cases} 1, \text{ if initial task is mapped with a } VM \\ 0, \text{ otherwise.} \end{cases} \quad (22)$$

The objective is to discover an optimum mapping for the n task to VM_1 to minimize the maximal Makespan. The problems are represented by $P||C_{\max}$, whereas P represent identical parallel VM_1 . The task is limited to 39, and the aim is to attain the minimal length of schedule

$$\min C \quad (23)$$

Subjected to

$$\sum_{VM=1}^1 = x_{ij}, 1 \leq \Pi \leq Task_J, \quad (24)$$

$$C - \sum_{task=1}^{Task_J} P_{task}^{x_i} \geq 0, 1 \leq VM \leq VM_1, \quad (25)$$

whereas the optimum values of C are $C_{\max}$ as well $x_l = 1$ while the tasks are mapped into VM; or else, $x_{IJ} = 0$.

4 Experimental validation

This section examines the performance of the QOAO-TS technique with recent approaches. The results are investigated in terms of five different measures, namely, makespan, throughput, flowtime, lateness, and utilization ratio, as shown in Table 1.

Table 1 Comparative result analysis of the QOAO-TS with existing techniques

Algorithm	Makespan $C_{\max}$	Throughput	Flow time	Lateness	Utilization ratio (%)
LPT	32.00	130.83	26.18	-60	13.79
LPT-CPSO	24.00	236.78	73.19	-42	12.52
SPT	39.00	053.81	10.68	-18	19.39
SPT-CPSO	08.00	160.52	32.06	-47	13.16
ECT	59.00	057.63	11.47	-18	18.76
ECT-CPSO	14.00	085.46	17.08	-91	15.79
EST	38.00	069.36	13.94	-17	17.40
EST-CPSO	05.00	114.86	22.92	-61	13.75
DeadLine	37.00	067.75	13.55	-18	17.89
DeadLine-CPSO	32.00	098.48	17.89	77	15.86
DueDate	37.00	067.65	13.49	-18	17.90
DueDate-CPSO	36.00	088.56	17.81	4	16.02
QOAO-TS	03.00	250.53	08.62	-92	20.59

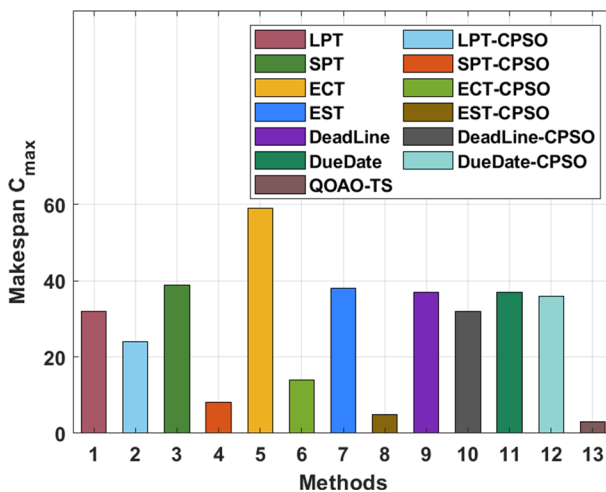
**Fig. 2** Makespan analysis of the QOAO-TS model with existing approaches

Figure 2 shows the makespan analysis of the QOAO-TS technique with existing techniques. The figure shows that the ECT, SPT, EST, deadline, DueData, DueDate-CPSO, LPT, and DeadLine-CPSO techniques attained higher makespans of 59, 39, 38, 37, 37, 36, 32, and 32, respectively. In addition, the LPT-CPSO and SPT-CPSO techniques have reached moderate makespans of 24 and 14, respectively. Although the SPT-CPSO and EST-CPSO techniques have accomplished somewhat reduced makespans of 8 and 5, the proposed QOAO-TS technique has demonstrated effective outcomes with a minimum makespan of 3.

The throughput analysis of the QOAO-TS manner with state-of-art techniques takes place in Fig. 3. The outcomes demonstrated that the SPT, ECT, DueDate-CPSO, DeadLine, EST, and ECT-CPSO manners resulted in lower throughputs of 53.81, 57.63, 67.65, 67.75, 69.36, and 85.46, respectively. Additionally, DueDate-CPSO, DeadLine-CPSO, EST-CPSO, LPT, SPT-CPSO, and LPT-CPSO have achieved managed throughputs of 88.56, 98.48, 114.86, 130.83, 160.52, and 236.78, respectively. Finally, the proposed QOAO-TS technique outperformed higher outcomes with a higher throughput of 250.53.

Figure 4 depicts the flow time analysis of the QOAO-TS algorithm with existing models. The figure demonstrates that the LPT-CPSO, SPT-CPSO, LPT, EST-CPSO, DeadLine-CPSO, DueDate-CPSO, ECT-CPSO, AND EST methods have obtained maximum flow times of 73.19, 32.06, 26.18, 22.92, 17.89, 17.81, 17.08, and 13.94, respectively. In addition, the DeadLine and DueDate techniques reached moderate flow times of 13.55 and 13.49, respectively. Although the ECT and SPT techniques have accomplished somewhat reduced flow times of 11.47 and 10.68, the proposed QOAO-TS technique has demonstrated effective outcomes with a minimum flow time of 8.62.

Figure 5 shows the lateness analysis of the QOAO-TS technique with existing techniques. The figure shows that the DeadLine-CPSO, DueDate-CPSO, EST, SPT-CPSO, ECT, DeadLine, DueDate, and LPT-CPSO techniques attained higher lateness of 77, 4, -17, -18, -18, -18, -18, and -42, respectively. In addition, the SPT-CPSO and LPT techniques have gained moderate lateness of -47 and -60, respectively. Likewise, the EST-CPSO and EST-CPSO methods have accomplished somewhat reduced lateness of -61 and -91, and the presented QOAO-TS technique has exhibited effective outcomes with a minimal lateness of -92.

The utilization ratio analysis of the QOAO-TS technique with recent methods takes place in Fig. 6. The results depicted that the DeadLine-CPSO, ECT-CPSO,

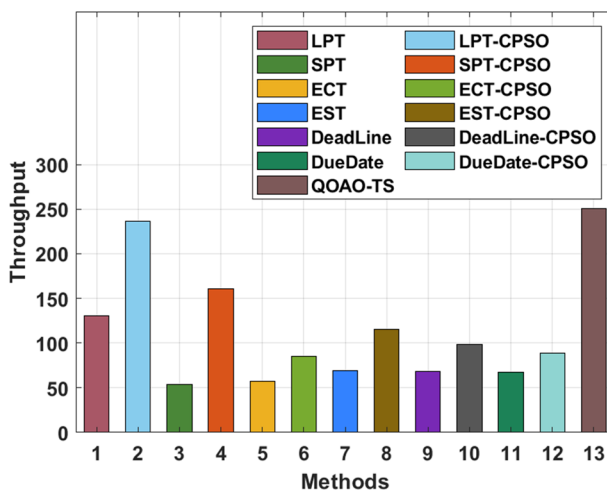


Fig. 3 Throughput analysis of the QOAO-TS model with existing approaches

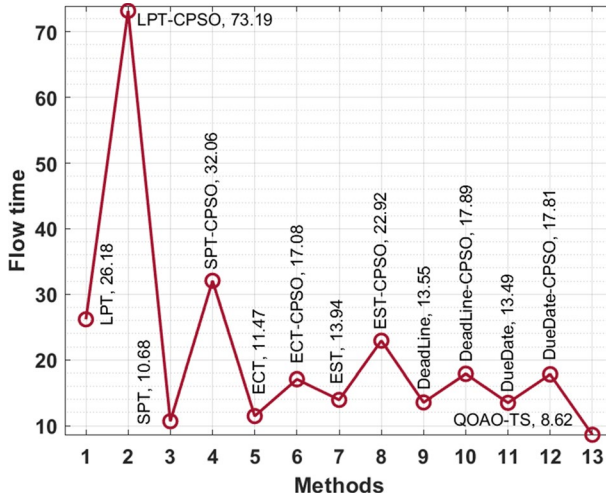


Fig. 4 Flow time analysis of the QOAO-TS model with existing approaches

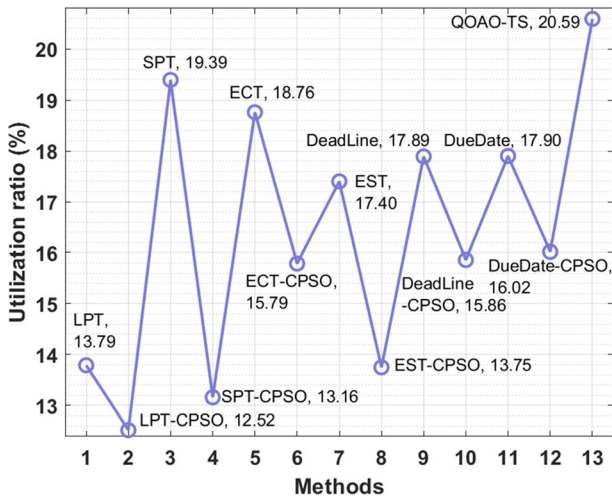


Fig. 5 Utilization ratio analysis of the QOAO-TS model with existing approaches

LPT, EST-CPSO, SPT-CPSO, and LPT-CPSO techniques have resulted in a lower utilization ratio of 15.86%, 15.79%, 13.79%, 13.75%, 13.16%, and 12.52% respectively. Moreover, SPT, ECT, DueDate, DeadLine, EST, and DueDate-CPSO have accomplished to a managed utilization ratio of 19.39%, 18.76%, 17.9%, 17.89%, 17.4%, and 16.02% respectively. However, the proposed QOAO-TS technique has demonstrated superior outcomes with a higher utilization ratio of 20.59%. By looking into the above-mentioned tables and figures, it is evident that the

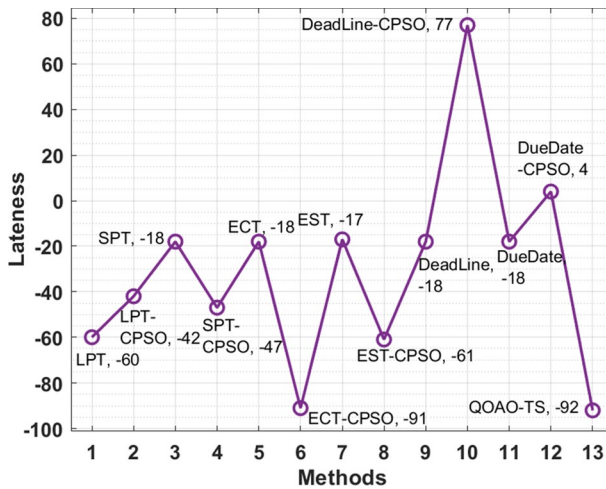


Fig. 6 Lateness analysis of the QOAO-TS model with existing approaches

presented QOAO-TS technique can be utilized as an effective tool for task scheduling in IoT cloud environment.

By looking at the abovementioned tables and figures, it is evident that the presented QOAO-TS technique can be utilized as an effective tool for task scheduling in an IoT cloud environment.

5 Conclusion

In this paper, a new QOAO-TS technique is designed to schedule tasks in an IoT-enabled cloud environment in an optimal way. The QOAO-TS technique is derived based on the integration of classical AO with the QOBL and thereby improves the outcome of the AO. In addition, an objective function is formulated in such a way that the makespan can be reduced with the optimum task scheduling process. Moreover, the QOAO-TS technique has utilized the relativity among the task scheduling and client need satisfaction by minimizing the makespan. To showcase the improved performance of the QOAO-TS technique over the other existing techniques, a detailed comparative analysis is carried out. The resultant outcomes stated the better performance of the QOAO-TS technique compared to recent approaches in terms of different evaluation measures. In the future, the hybridization of two metaheuristic algorithms and deep learning models can be presented to optimally schedule the tasks in the IoT cloud environment.

Funding The authors received no specific funding for this study.

Declarations

Conflict of interest The authors declare that they have no conflicts of interest to report regarding the present study.

References

1. Nguyen BM, Thi Thanh Binh H, Do Son B (2019) Evolutionary algorithms to optimize task scheduling problem for the IoT based bag-of-tasks application in cloud-fog computing environment. *Appl Sci* 9(9):1730
2. Fan J, Wei X, Wang T, Lan T, and Subramaniam S (2017) December. Deadline-aware task scheduling in a tiered IoT infrastructure. In *GLOBECOM 2017–2017 IEEE Global Communications Conference* (pp. 1–7). IEEE
3. Abdelmoneem RM, Benslimane A, Shaaban E (2020) Mobility-aware task scheduling in cloud-Fog IoT-based healthcare architectures. *Comput Netw* 179:107348
4. Huang J, Li S, Chen Y (2020) Revenue-optimal task scheduling and resource management for IoT batch jobs in mobile edge computing. *Peer-to-Peer Netw Appl* 13(5):1776–1787
5. He Z, Zhang Y, Tak B, Peng L (2019) Green fog planning for optimal internet-of-thing task scheduling. *IEEE Access* 8:1224–1234
6. Zhou J, Sun J, Cong P, Liu Z, Zhou X, Wei T, Hu S (2019) Security-critical energy-aware task scheduling for heterogeneous real-time MPSoCs in IoT. *IEEE Trans Serv Comput* 13(4):745–758
7. Zhang G, Shen F, Zhang Y, Yang R, Yang Y, and Jorswieck EA (2018) October. Delay minimized task scheduling in fog-enabled IoT networks. In *2018 10th International Conference on Wireless Communications and Signal Processing (WCSP)* (pp. 1–6). IEEE
8. Gedawy H, Habak K, Harras KA, and Hamdi M (2018) Awakening the cloud within: Energy-aware task scheduling on edge IoT devices. In *2018 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)* (pp. 191–196). IEEE
9. Fellir F, El Attar A, Nafil K, and Chung L (2020) A multi-Agent based model for task scheduling in cloud-fog computing platform. In *2020 IEEE International Conference on Informatics, IoT, and Enabling Technologies (ICIoT)* (pp. 377–382). IEEE
10. Sheng S, Chen P, Chen Z, Wu L, Yao Y (2021) Deep reinforcement learning-based task scheduling in IoT edge computing. *Sensors* 21(5):1666
11. Hasan MZ, Al-Rizzo H (2020) Task scheduling in Internet of Things cloud environment using a robust particle swarm optimization. *Concurrency Comput Pract Exp* 32(2):e5442
12. Abdel-Basset M, Mohamed R, Elhoseny M, Bashir AK, Jolfaei A, Kumar N (2020) Energy-aware marine predators algorithm for task scheduling in IoT-based fog computing applications. *IEEE Trans Industr Inf* 17(7):5068–5076
13. Al-Turjman F, Hasan MZ, Al-Rizzo H (2019) Task scheduling in cloud-based survivability applications using swarm optimization in IoT. *Transactions Emerg Telecommun Technol* 30(8):e3539
14. Ma X, Gao H, Xu H, Bian M (2019) An IoT-based task scheduling optimization scheme considering the deadline and cost-aware scientific workflow for cloud computing. *EURASIP J Wirel Commun Netw* 2019(1):1–19
15. Basu S, Karuppiiah M, Selvakumar K, Li KC, Islam SH, Hassan MM, Bhuiyan MZA (2018) An intelligent/cognitive model of task scheduling for IoT applications in cloud computing environment. *Futur Gener Comput Syst* 88:254–261
16. Liu T, Duan G (2021) Task allocation optimization model in mechanical product development based on Bayesian network and ant colony algorithm. *J Supercomput* 77:13963–13991. <https://doi.org/10.1007/s11227-021-03831-3>
17. Abualigah L, Alkhrahsheh M (2021) Amended hybrid multi-verse optimizer with genetic algorithm for solving task scheduling problem in cloud computing. *J Supercomput*. <https://doi.org/10.1007/s11227-021-03915-0>
18. Li C, Zhang J, Tang H (2019) Replica-aware task scheduling and load balanced cache placement for delay reduction in multi-cloud environment. *J Supercomput* 75:2805–2836. <https://doi.org/10.1007/s11227-018-2695-9>

19. Chen R, Chen X, Yang C (2021) Using a task dependency job-scheduling method to make energy savings in a cloud computing environment. *J Supercomput*. <https://doi.org/10.1007/s11227-021-04035-5>
20. Abualigah L, Yousri D, Abd Elaziz M, Ewees AA, Al-Qaness MA, Gandomi AH (2021) Aquila optimizer: a novel meta-heuristic optimization algorithm. *Comput Industrial Eng* 157:107250

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

**M. Kandan¹ · Anbazhagan Krishnamurthy² · S. Arun Mozhi Selvi³ ·
Mohamed Yacin Sikkandar⁴ · Mohamed Abdelkader Aboamer⁴ · T. Tamilvizhi⁵**

Anbazhagan Krishnamurthy
rishianbu@gmail.com

S. Arun Mozhi Selvi
drarunmozhiselvi@gmail.com

Mohamed Yacin Sikkandar
m.sikkandar@mu.edu.sa

Mohamed Abdelkader Aboamer
m.aboamer@mu.edu.da

T. Tamilvizhi
tamivizhi.phd.it@gmail.com

¹ Department of CSE, Aditya Engineering College, Surempalem, Andhra Pradesh 533437, India

² Department of CSE, Velammal Institute of Technology, Chennai, Tamilnadu 601204, India

³ Holycross Engineering College, Thoothukudi, Tamilnadu 628851, India

⁴ Department of Medical Equipment Technology, College of Applied Medical Sciences, Majmaah University, Al Majmaah 11952, Saudi Arabia

⁵ Department of Information Technology, Vel Tech Multi Tech Dr. Rangarajan Dr. Sakunthala Engineering College, Chennai, India

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH (“Springer Nature”).

Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users (“Users”), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use (“Terms”). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.

These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.

We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.

While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval, sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.

These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.

To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.

Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.

If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com